

Informe Triggers Módulo 6
Datos Financieros (NIC 41)

Versión: 1.1

16 de mayo de 2026

Juan David Peña Roldán

Universidad Surcolombiana

Neiva, Huila

2026

TGR-M06-01: Inmutabilidad de registros confirmados en reconocimientos_iniciales

Descripción	Impide cualquier UPDATE o DELETE sobre un reconocimiento inicial que ya tiene estado CONFIRMADO o REVERTIDO. Evalúa el estado del registro antes del UPDATE/DELETE. Si el estado es CONFIRMADO o REVERTIDO, lanza una excepción y cancela la operación. Permite modificaciones solo sobre registros en estado PENDIENTE_CONFIRMACION.
Aplicado a	RF-82
Restricción origen RF	RF-82 — Restricciones, punto 2: <i>"El reconocimiento inicial confirmado es un registro inmutable: no puede editarse ni eliminarse directamente."</i> y RNF-82-03: <i>"El registro del reconocimiento inicial es inmutable; el sistema debe impedir cualquier modificación o eliminación directa."</i>
Justificación	La inmutabilidad de un asiento contable confirmado es una garantía de integridad que no puede depender de la capa de aplicación. Cualquier acceso directo a la BD (herramientas de administración, migraciones, bugs) podría romperla si solo está en el backend.

```
CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_01_inmutabilidad_reconocimiento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Para UPDATE: verificar el estado ANTES del cambio (OLD)
    IF TG_OP = 'UPDATE' THEN
        IF OLD.estado IN ('CONFIRMADO', 'REVERTIDO') THEN
```

```

        RAISE EXCEPTION
        '[TGR-M06-01] El reconocimiento inicial id=% tiene
estado % y es inmutable. '
        'No se permiten modificaciones directas. Para
anular un reconocimiento confirmado '
        'utilice el mecanismo de reversión auditada
(revisiones_reconocimiento). '
        'Requerimiento RF-82, Restricción 2.',
        OLD.id_reconocimiento_inicial, OLD.estado
    USING ERRCODE = 'P0001';
END IF;
END IF;

-- Para DELETE: ningún reconocimiento puede eliminarse
independientemente del estado
IF TG_OP = 'DELETE' THEN
    RAISE EXCEPTION
        '[TGR-M06-01] El reconocimiento inicial id=% no puede
eliminarse. '
        'Los reconocimientos son registros permanentes del
libro mayor. '
        'Estado actual: %. Requerimiento RF-82, Restricción 2
y RNF-82-03.',
        OLD.id_reconocimiento_inicial, OLD.estado
    USING ERRCODE = 'P0001';
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_01_inmutabilidad_reconocimiento
BEFORE UPDATE OR DELETE
ON modulo6.reconocimientos_iniciales
FOR EACH ROW
EXECUTE FUNCTION
modulo6.fn_tgr_m06_01_inmutabilidad_reconocimiento();

COMMENT ON TRIGGER tgr_m06_01_inmutabilidad_reconocimiento
ON modulo6.reconocimientos_iniciales
IS 'TGR-M06-01 | RF-82 Restricción 2 y RNF-82-03 | Garantiza
la inmutabilidad de '

```

```
'reconocimientos en estado CONFIRMADO o REVERTIDO. Bloquea cualquier UPDATE o DELETE '
'directo sobre estos registros. Solo permite modificaciones sobre estado PENDIENTE_CONFIRMACION.';
```

TGR-M06-02: Unicidad de reconocimiento inicial vigente por activo biológico

Descripción	Garantiza que no pueda existir más de un reconocimiento inicial en estado CONFIRMADO para el mismo activo biológico, incluso si el constraint UNIQUE de la tabla pudiera eludirse mediante estados intermedios. Antes de INSERT, verifica si ya existe un reconocimiento en estado CONFIRMADO para el mismo <code>id_activo_biologico</code>. Si existe, lanza excepción. Permite INSERT si el anterior está REVERTIDO o PENDIENTE_CONFIRMACION (el backend debería limpiar el pendiente antes).
Aplicado a	RF-82
Restricción origen RF	RF-82 — Restricciones, punto 3: <i>"No se puede registrar un segundo reconocimiento inicial para un activo biológico que ya tenga uno vigente en M06."</i> y FA-1: <i>"Si el id_activo_biologico ya posee un reconocimiento inicial en M06, el sistema rechaza la operación."</i>
Justificación	El constraint UNIQUE existente en <code>uq_reconocimientos_iniciales_id_activo_biologico</code> cubre el caso general, pero no distingue por estado. Un activo REVERTIDO debería poder tener un nuevo reconocimiento. Este trigger cubre la semántica de negocio: solo bloquea cuando ya existe uno CONFIRMADO.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_02_unicidad_reconocimiento_confirmado()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_reconocimiento_existente INTEGER;
BEGIN
    -- Solo aplica en INSERT o cuando el estado cambia a CONFIRMADO
    IF TG_OP = 'INSERT' OR (TG_OP = 'UPDATE' AND NEW.estado =
'CONFIRMADO' AND OLD.estado != 'CONFIRMADO') THEN

        SELECT id_reconocimiento_inicial
        INTO v_reconocimiento_existente
        FROM modulo6.reconocimientos_iniciales
        WHERE id_actvo_biologico = NEW.id_actvo_biologico
            AND estado = 'CONFIRMADO'
            AND id_reconocimiento_inicial !=
COALESCE(NEW.id_reconocimiento_inicial, -1)
        LIMIT 1;

        IF FOUND THEN
            RAISE EXCEPTION
                '[TGR-M06-02] El activo biológico id=% ya posee un
reconocimiento inicial '
                'CONFIRMADO (id_reconocimiento=%). No se puede registrar
un segundo reconocimiento '
                'vigente. Requerimiento RF-82, Restricción 3 y FA-1.',
                NEW.id_actvo_biologico, v_reconocimiento_existente
            USING ERRCODE = 'P0001';
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_02_unicidad_reconocimiento_confirmado
BEFORE INSERT OR UPDATE OF estado
ON modulo6.reconocimientos_iniciales
FOR EACH ROW
EXECUTE FUNCTION
modulo6.fn_tgr_m06_02_unicidad_reconocimiento_confirmado();

COMMENT ON TRIGGER tgr_m06_02_unicidad_reconocimiento_confirmado
ON modulo6.reconocimientos_iniciales

```

```
IS 'TGR-M06-02 | RF-82 Restricción 3 y FA-1 | Impide que exista más
de un reconocimiento '
    'inicial en estado CONFIRMADO para el mismo activo biológico.
Distingue por estado, '
    'permitiendo un nuevo reconocimiento solo si el anterior fue
REVERTIDO.';
```

TGR-M06-03: Inmutabilidad de revisiones de reconocimiento (reversiones)

Descripción	Impide UPDATE y DELETE sobre cualquier registro de reversión en revisiones_reconocimiento. Una reversión auditada es permanente e inmutable. Bloquea cualquier UPDATE o DELETE sobre revisiones_reconocimiento. No hay excepciones: una reversión nunca puede modificarse ni eliminarse.
Aplicado a	RF-82
Restricción origen RF	RF-82 — Restricciones, punto 2f: <i>"Trazabilidad: el asiento de reversión queda vinculado al id_reconocimiento original... Ambos registros (el original y la reversión) son inmutables y coexisten en el historial contable."</i> y punto 2g: <i>"El asiento de reversión no elimina el reconocimiento original del historial."</i>
Justificación	Las reversiones son evidencia contable auditada. Su inmutabilidad debe estar garantizada a nivel de motor de base de datos, no solo por restricciones de aplicación.

```
CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_03_inmutabilidad_revisiones()
RETURNS TRIGGER
```

```

LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M06-03] La reversión id=% es un registro de auditoría
inmutable y no puede '
            'modificarse. Los asientos de reversión coexisten
permanentemente con el reconocimiento '
            'original para garantizar trazabilidad NIC 41. Requerimiento
RF-82, Restricción 2f.',
            OLD.id_revision_reconocimiento
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M06-03] La reversión id=% no puede eliminarse. Es
evidencia contable auditada '
            'que debe coexistir con el reconocimiento original id=% en
el historial. '
            'Requerimiento RF-82, Restricción 2g.',
            OLD.id_revision_reconocimiento,
            OLD.id_reconocimiento_revertido
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_03_inmutabilidad_revisiones
    BEFORE UPDATE OR DELETE
    ON modulo6.revisiones_reconocimiento
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_03_inmutabilidad_revisiones();

COMMENT ON TRIGGER tgr_m06_03_inmutabilidad_revisiones
    ON modulo6.revisiones_reconocimiento
    IS 'TGR-M06-03 | RF-82 Restricciones 2f y 2g | Garantiza
inmutabilidad absoluta de los '
        'asientos de reversión. Ningún UPDATE o DELETE está permitido
sobre esta tabla. '
        'Las revisiones son evidencia contable permanente.';

```

TGR-M06-04: Consistencia matemática en mediciones posteriores

Descripción	Verifica que la ecuación de consistencia se cumpla antes de insertar o actualizar una medición posterior: $\text{variacion_total} = \text{ganancia_perdida_transformacion} + \text{ganancia_perdida_precio_mercado}$ y $\text{variacion_total} = \text{valor_razonable_actual} - \text{valor_razonable_anterior}$. Calcula y compara las dos ecuaciones de consistencia con tolerancia de ± 0.01 COP para manejo de redondeo de decimales. Si alguna no cuadra, lanza excepción detallada indicando los valores recibidos.
Aplicado a	RF-83
Restricción origen RF	RF-83 — RNF-83-05: "Los componentes de transformación biológica y variación de precio deben sumar exactamente la variación total del valor razonable; el sistema debe validar esta consistencia antes de confirmar el registro." y CA-6: "El sistema garantiza que la suma de los componentes (transformación + precio) es exactamente igual a la variación total del valor razonable."
Justificación	Es una invariante matemática sobre datos que residen en la misma fila. La BD puede verificarla atómicamente en el momento de la escritura, sin depender del backend. Previene inconsistencias que llegarían a estados financieros incorrectos.

```
CREATE OR REPLACE FUNCTION modulo6.fn_tgr_m06_04_consistencia_medicion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
```



```

DECLARE
    v_variacion_calculada    NUMERIC(18,4);
    v_suma_componentes       NUMERIC(18,4);
    v_tolerancia             NUMERIC(18,4) := 0.01;
BEGIN
    -- Ecuación 1: variacion_total debe ser igual a la diferencia de
    valores razonables
    v_variacion_calculada := NEW.valor_razonable_actual -
    NEW.valor_razonable_anterior;

    IF ABS(NEW.variacion_total - v_variacion_calculada) > v_tolerancia
    THEN
        RAISE EXCEPTION
            '[TGR-M06-04] Inconsistencia en medición posterior:
variacion_total=% '
            'no coincide con (valor_razonable_actual % -
valor_razonable_anterior %) = %. '
            'Diferencia: %. Tolerancia máxima: %. Requerimiento RF-83,
RNF-83-05.',
            NEW.variacion_total,
            NEW.valor_razonable_actual,
            NEW.valor_razonable_anterior,
            v_variacion_calculada,
            ABS(NEW.variacion_total - v_variacion_calculada),
            v_tolerancia
        USING ERRCODE = 'P0001';
    END IF;

    -- Ecuación 2: variacion_total debe ser igual a la suma de los dos
    componentes
    v_suma_componentes := NEW.ganancia_perdida_transformacion +
    NEW.ganancia_perdida_precio_mercado;

    IF ABS(NEW.variacion_total - v_suma_componentes) > v_tolerancia THEN
        RAISE EXCEPTION
            '[TGR-M06-04] Inconsistencia en medición posterior:
variacion_total=% '
            'no coincide con (ganancia_perdida_transformacion % +
ganancia_perdida_precio_mercado %) = %. '
            'Diferencia: %. Tolerancia máxima: %. Requerimiento RF-83,
RNF-83-05.',
            NEW.variacion_total,
            NEW.ganancia_perdida_transformacion,
            NEW.ganancia_perdida_precio_mercado,
            v_suma_componentes,
            ABS(NEW.variacion_total - v_suma_componentes),

```

```

        v_tolerancia
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_04_consistencia_medicion
    BEFORE INSERT OR UPDATE
    ON modulo6.mediciones_posteriores
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_04_consistencia_medicion();

COMMENT ON TRIGGER tgr_m06_04_consistencia_medicion
    ON modulo6.mediciones_posteriores
    IS 'TGR-M06-04 | RF-83 RNF-83-05 y CA-6 | Valida que variacion_total
= '
    'ganancia_perdida_transformacion +
ganancia_perdida_precio_mercado, y que '
    'variacion_total = valor_razonable_actual -
valor_razonable_anterior. '
    'Tolerancia de ±0.01 COP para redondeo.';

```

TGR-M06-05: Inmutabilidad de mediciones posteriores confirmadas

Descripción	Impide UPDATE y DELETE sobre mediciones en estado COMPLETADO. Las mediciones confirmadas son registros auditables e inmutables. Bloquea UPDATE y DELETE sobre mediciones en estado COMPLETADO. Permite modificaciones solo sobre estados MEDICION_PENDIENTE o ERROR, que son estados de trabajo del backend.
Aplicado a	RF-83
Restricción origen RF	RF-83 — RNF-83-04: <i>"Los registros de medición son inmutables una vez confirmados; solo pueden complementarse"</i>

	<i>con mediciones posteriores o con asientos de reversión auditados."</i>
Justificación	Al igual que los reconocimientos, las mediciones son parte del libro mayor NIC 41. Su inmutabilidad debe ser una garantía de la BD.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_05_inmutabilidad_medicion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M06-05] La medición posterior id=% no puede
eliminar. '
            'Es un registro inmutable del libro mayor NIC 41. Estado: %.
',
            'Requerimiento RF-83, RNF-83-04.',
            OLD."id_ medicion_posterior", OLD.estado
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'UPDATE' AND OLD.estado = 'COMPLETADO' THEN
        RAISE EXCEPTION
            '[TGR-M06-05] La medición posterior id=% tiene estado
COMPLETADO y es inmutable. '
            'No se permiten modificaciones directas sobre mediciones
confirmadas. '
            'Requerimiento RF-83, RNF-83-04.',
            OLD."id_ medicion_posterior"
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_05_inmutabilidad_medicion
BEFORE UPDATE OR DELETE
ON modulo6.mediciones_posteriores
FOR EACH ROW
EXECUTE FUNCTION modulo6.fn_tgr_m06_05_inmutabilidad_medicion();

```

```

COMMENT ON TRIGGER tgr_m06_05_inmutabilidad_medicion
ON modulo6.mediciones_posteriores
IS 'TGR-M06-05 | RF-83 RNF-83-04 | Garantiza inmutabilidad de
mediciones en estado '
'COMPLETADO. Bloquea todos los DELETE independientemente del
estado. Permite UPDATE '
'solo sobre estados MEDICION_PENDIENTE o ERROR.';

```

TGR-M06-06: Valor neto no negativo y clasificación de costos en cálculos de valor razonable

Descripción	Garantiza que el valor_neto no supere el valor_razonable_bruto de forma inconsistente, y que los costos incluidos en el cálculo sean todos positivos o cero (nunca negativos). Verifica que cada componente de costo sea ≥ 0, que el valor_neto sea igual a valor_razonable_bruto - suma_costos con tolerancia de ± 0.01, y que el valor_razonable_bruto sea > 0.
Aplicado a	RF-84
Restricción origen RF	RF-84 — Restricciones, punto 3: "El valor razonable menos costos de venta puede ser menor o igual a cero; en estos casos el sistema debe generar una alerta crítica... si el cálculo arroja un resultado negativo, el sistema alerta al Contador para revisión de los parámetros de costos." y punto 6: "Todo costo utilizado en el cálculo debe estar explícitamente clasificado como tipo_costo = VENTA." También CA-1: validación del cálculo valor_razonable - costos_de_venta.

Justificación	La consistencia aritmética entre el valor bruto, los costos y el valor neto es verificable en la BD de forma atómica. Los costos negativos nunca deben existir en el modelo financiero.
----------------------	---

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_06_consistencia_calculo_vr()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_suma_costos          NUMERIC(18,4);
    v_valor_netto_calc     NUMERIC(18,4);
    v_tolerancia           NUMERIC(18,4) := 0.01;
BEGIN
    -- Validar que el valor razonable bruto sea positivo
    IF NEW.valor_razonable_bruto <= 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-06] valor_razonable_bruto=% debe ser mayor a cero.
            '
            'El precio de mercado de referencia no puede ser cero ni
            negativo. '
            'Requerimiento RF-84, CA-1.',
            NEW.valor_razonable_bruto
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar que ningún componente de costo sea negativo
    IF NEW.costo_transporte < 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-06] costo_transporte=% no puede ser negativo. '
            'Los costos de venta son valores positivos o cero.
            Requerimiento RF-84.',
            NEW.costo_transporte
        USING ERRCODE = 'P0001';
    END IF;

    IF NEW.costo_comisiones < 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-06] costo_comisiones=% no puede ser negativo. '
            'Requerimiento RF-84.',
            NEW.costo_comisiones
        USING ERRCODE = 'P0001';
    END IF;

```

```

    IF NEW.costo_impuestos_transaccion < 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-06] costo_impuestos_transaccion=% no puede ser
negativo. '
            'Requerimiento RF-84.',
            NEW.costo_impuestos_transaccion
        USING ERRCODE = 'P0001';
    END IF;

    IF NEW.otros_costos_disposicion < 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-06] otros_costos_disposicion=% no puede ser
negativo. '
            'Requerimiento RF-84.',
            NEW.otros_costos_disposicion
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar consistencia aritmética: valor_neto = valor_bruto -
suma_costos
    v_suma_costos := NEW.costo_transporte
        + NEW.costo_comisiones
        + NEW.costo_impuestos_transaccion
        + NEW.otros_costos_disposicion;

    v_valor_neto_calc := NEW.valor_razonable_bruto - v_suma_costos;

    IF ABS(NEW.valor_neto - v_valor_neto_calc) > v_tolerancia THEN
        RAISE EXCEPTION
            '[TGR-M06-06] Inconsistencia aritmética: valor_neto=% no
coincide con '
            '(valor_razonable_bruto % - suma_costos %) = %. '
            'Diferencia: %. Tolerancia: %. Requerimiento RF-84, CA-1.',
            NEW.valor_neto,
            NEW.valor_razonable_bruto,
            v_suma_costos,
            v_valor_neto_calc,
            ABS(NEW.valor_neto - v_valor_neto_calc),
            v_tolerancia
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

```

```

CREATE OR REPLACE TRIGGER tgr_m06_06_consistencia_calculo_vr
  BEFORE INSERT OR UPDATE
  ON modulo6.calculos_valor_razonable
  FOR EACH ROW
  EXECUTE FUNCTION modulo6.fn_tgr_m06_06_consistencia_calculo_vr();

COMMENT ON TRIGGER tgr_m06_06_consistencia_calculo_vr
  ON modulo6.calculos_valor_razonable
  IS 'TGR-M06-06 | RF-84 Restricciones 3 y 6, CA-1 | Valida: (1)
valor_razonable_bruto > 0, '
      '(2) todos los componentes de costo >= 0, (3) valor_netto =
valor_bruto - suma_costos '
      'con tolerancia ±0.01 COP.';

```

TGR-M06-07: Inmutabilidad del repositorio de precios de mercado (append-only)

Descripción	Impide UPDATE y DELETE sobre cualquier registro de precios_mercado. El repositorio es estrictamente append-only. Bloquea absolutamente cualquier UPDATE o DELETE sobre precios_mercado. El mecanismo de corrección es siempre un nuevo INSERT con id_precio_coregido apuntando al registro que se desea corregir.
Aplicado a	RF-89
Restricción origen RF	RF-89 — Restricciones: "El historial de precios es append-only: ninguna entrada puede ser editada ni eliminada bajo ninguna circunstancia." y FA-6: "El sistema bloquea la operación sin modificar ningún dato, notifica al Contador que el historial de precios es inmutable." y CA-6: "El historial de precios es inmutable: no existe ninguna operación de edición ni eliminación de registros existentes."

Justificación	La inmutabilidad de un repositorio de precios que sustenta todos los cálculos de valor razonable es una garantía de auditoría crítica bajo NIC 41. Sin esta protección en BD, cualquier corrección accidental o maliciosa de precios históricos distorsionaría los estados financieros retroactivamente.
----------------------	--

```

CREATE OR REPLACE FUNCTION modulo6.fn_tgr_m06_07_inmutabilidad_precios()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M06-07] El precio de mercado id=% no puede
modificarse. '
            'El repositorio de precios es append-only e inmutable. '
            'Para corregir un precio erróneo, registre un nuevo precio
con idprecio_coregido=% '
            'y la fecha_vigencia correspondiente. Requerimiento RF-89,
Restricciones y FA-6.',
            OLD.idprecio_mercado, OLD.idprecio_mercado
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M06-07] El precio de mercado id=% no puede eliminarse.
'
            'El historial de precios es inmutable y permanente. '
            'Los registros históricos sustentan los cálculos de valor
razonable auditados. '
            'Requerimiento RF-89, Restricciones y CA-6.',
            OLD.idprecio_mercado
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_07_inmutabilidad_precios
BEFORE UPDATE OR DELETE
ON modulo6.precios_mercado

```



```

FOR EACH ROW
EXECUTE FUNCTION modulo6.fn_tgr_m06_07_inmutabilidad_precios();

COMMENT ON TRIGGER tgr_m06_07_inmutabilidad_precios
ON modulo6.precios_mercado
IS 'TGR-M06-07 | RF-89 Restricciones (append-only), FA-6, CA-6 |
Garantiza que ningún '
    'precio de mercado pueda ser editado o eliminado. El mecanismo de
corrección '
    'es exclusivamente un nuevo INSERT con id_precio_coregido
referenciando el original.';

```

TGR-M06-08: Validación de fecha_vigencia en precios de mercado

Descripción	Garantiza que fecha_vigencia no sea anterior en más de 365 días a la fecha actual del sistema, y que si existe fecha_vencimiento, esta sea posterior a fecha_vigencia. Al INSERT de un nuevo precio, verifica que fecha_vigencia >= CURRENT_DATE - INTERVAL '365 days' y que si fecha_vencimiento no es nula, sea estrictamente mayor que fecha_vigencia.
Aplicado a	RF-89
Restricción origen RF	RF-89 — Restricciones: "La <i>fecha_vigencia</i> no puede ser anterior en más de 365 días a la fecha actual del sistema (para evitar cargas históricas accidentales que afecten cálculos retroactivos)." y "si se ingresó fecha de vencimiento, esta es posterior a la fecha de vigencia."
Justificación	Son validaciones de fechas sobre columnas de la misma fila, verificables sin lógica de negocio compleja. La BD es el lugar correcto para proteger la integridad temporal del repositorio de precios.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_08_validacion_fechas_precio()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Validar que fecha_vigencia no sea anterior en más de 365 días
    IF NEW.fecha_vigencia < (CURRENT_DATE - INTERVAL '365 days') THEN
        RAISE EXCEPTION
            '[TGR-M06-08] fecha_vigencia=% es anterior en más de 365
días a la fecha actual (%). '
            'No se admiten cargas históricas que podrían afectar
cálculos retroactivos. '
            'Requerimiento RF-89, Restricciones.',
            NEW.fecha_vigencia, CURRENT_DATE
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar que fecha_vencimiento sea posterior a fecha_vigencia (si
se informa)
    IF NEW.fecha_vencimiento IS NOT NULL AND NEW.fecha_vencimiento <=
NEW.fecha_vigencia THEN
        RAISE EXCEPTION
            '[TGR-M06-08] fecha_vencimiento=% debe ser posterior a
fecha_vigencia=%. '
            'Un precio no puede vencer antes o en la misma fecha de su
entrada en vigencia. '
            'Requerimiento RF-89, Restricciones.',
            NEW.fecha_vencimiento, NEW.fecha_vigencia
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar que precio_unitario sea positivo
    IF NEW.precio_unitario <= 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-08] precio_unitario=% debe ser mayor a cero. '
            'No se admiten precios cero ni negativos. Requerimiento
RF-89, Restricciones y CA-3.',
            NEW.precio_unitario
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

```

```

CREATE OR REPLACE TRIGGER tgr_m06_08_validacion_fechas_precio
  BEFORE INSERT
  ON modulo6.precios_mercado
  FOR EACH ROW
  EXECUTE FUNCTION modulo6.fn_tgr_m06_08_validacion_fechas_precio();

COMMENT ON TRIGGER tgr_m06_08_validacion_fechas_precio
  ON modulo6.precios_mercado
  IS 'TGR-M06-08 | RF-89 Restricciones (fechas) y CA-3 | Valida en
INSERT: '
      '(1) fecha_vigencia no puede ser anterior en más de 365 días, '
      '(2) fecha_vencimiento debe ser posterior a fecha_vigencia si se
informa, '
      '(3) precio_unitario debe ser mayor a cero.';

```

TGR-M06-09: Inmutabilidad de registros en períodos contables cerrados

Descripción	Impide INSERT, UPDATE y DELETE sobre registros financieros que pertenecen a un período contable en estado CERRADO o EN_CIERRE en las tablas: variaciones_valor_razonable, mediciones_posteriores, reconocimientos_productos_agricolas y registros_costos. Para cada tabla listada, antes de INSERT/UPDATE/DELETE consulta el estado del id_periodo_contable asociado. Si es CERRADO o EN_CIERRE, bloquea la operación. Se implementa como función genérica parametrizable.
Aplicado a	RF-87, RF-86, RF-83, RF-85, RF-90
Restricción origen RF	RF-87 — Descripción: <i>"Una vez cerrado, ningún registro del periodo puede ser modificado."</i> RF-86 — Restricciones: <i>"Solo</i>

	<i>se permite registrar variaciones sobre activos biológicos en periodos contables con estado ABIERTO." RF-90 — Restricción 5.5: "Los registros del período quedan inmutables una vez cerrado el período contable en RF-87."</i>
Justificación	La inmutabilidad de un período cerrado es la garantía central del sistema financiero. Debe estar en la BD para que sea imposible modificar datos de períodos cerrados incluso desde herramientas de administración.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_09_periodo_cerrado_bloqueo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_periodo    modulo6.enum_periodo_contables_estado;
    v_id_periodo        INTEGER;
BEGIN
    -- Determinar el id_periodo_contable según la operación
    IF TG_OP = 'DELETE' THEN
        v_id_periodo := OLD.id_periodo_contable;
    ELSE
        v_id_periodo := NEW.id_periodo_contable;
    END IF;

    -- Consultar el estado del período
    SELECT estado
    INTO v_estado_periodo
    FROM modulo6.periodos_contables
    WHERE id_periodo_contable = v_id_periodo;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            '[TGR-M06-09] El período contable id=% no existe. '
            'No se puede registrar información sin un período contable
válido.',
            v_id_periodo
        USING ERRCODE = 'P0001';
    END IF;

    IF v_estado_periodo IN ('CERRADO', 'EN_CIERRE') THEN

```

```

        RAISE EXCEPTION
        '[TGR-M06-09] El período contable id=% tiene estado % y es
inmutable. '
        'No se pueden registrar, modificar ni eliminar registros
sobre períodos '
        'CERRADOS o EN_CIERRE. Las correcciones deben realizarse en
el período siguiente '
        'con referencia explícita al registro que se corrige. '
        'Tabla afectada: %. Operación: %. RF-87.',
        v_id_periodo, v_estado_periodo, TG_TABLE_NAME, TG_OP
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    END IF;

    RETURN NEW;
END;
$$;

-- Aplicar a variaciones_valor_razonable
CREATE OR REPLACE TRIGGER tgr_m06_09a_periodo_cerrado_variaciones
    BEFORE INSERT OR UPDATE OR DELETE
    ON modulo6.variaciones_valor_razonable
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_09_periodo_cerrado_bloqueo();

COMMENT ON TRIGGER tgr_m06_09a_periodo_cerrado_variaciones
    ON modulo6.variaciones_valor_razonable
    IS 'TGR-M06-09a | RF-87, RF-86 Restricciones | Bloquea DML sobre
variaciones de valor '
        'razonable cuando el período contable está CERRADO o EN_CIERRE.';

-- Aplicar a mediciones_posteriores
CREATE OR REPLACE TRIGGER tgr_m06_09b_periodo_cerrado_mediciones
    BEFORE INSERT OR UPDATE OR DELETE
    ON modulo6.mediciones_posteriores
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_09_periodo_cerrado_bloqueo();

COMMENT ON TRIGGER tgr_m06_09b_periodo_cerrado_mediciones
    ON modulo6.mediciones_posteriores
    IS 'TGR-M06-09b | RF-87, RF-83 | Bloquea DML sobre mediciones
posteriores cuando '
        'el período contable está CERRADO o EN_CIERRE.';

```

```

-- Aplicar a reconocimientos_productos_agricolas
CREATE OR REPLACE TRIGGER tgr_m06_09c_periodo_cerrado_productos
    BEFORE INSERT OR UPDATE OR DELETE
    ON modulo6.reconocimientos_productos_agricolas
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_09_periodo_cerrado_bloqueo();

COMMENT ON TRIGGER tgr_m06_09c_periodo_cerrado_productos
    ON modulo6.reconocimientos_productos_agricolas
    IS 'TGR-M06-09c | RF-87, RF-85 Restricción 5 | Bloquea DML sobre
reconocimientos de '
        'productos agrícolas cuando el período contable está CERRADO o
EN_CIERRE.';

-- Aplicar a registros_costos
CREATE OR REPLACE TRIGGER tgr_m06_09d_periodo_cerrado_costos
    BEFORE INSERT OR UPDATE OR DELETE
    ON modulo6.registros_costos
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_09_periodo_cerrado_bloqueo();

COMMENT ON TRIGGER tgr_m06_09d_periodo_cerrado_costos
    ON modulo6.registros_costos
    IS 'TGR-M06-09d | RF-87, RF-90 Restricción 5.5 | Bloquea DML sobre
registros de costos '
        'cuando el período contable está CERRADO o EN_CIERRE.';

-- Aplicar a reconocimientos_iniciales
CREATE OR REPLACE TRIGGER tgr_m06_09e_periodo_cerrado_reconocimientos
    BEFORE INSERT OR UPDATE OR DELETE
    ON modulo6.reconocimientos_iniciales
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_09_periodo_cerrado_bloqueo();

COMMENT ON TRIGGER tgr_m06_09e_periodo_cerrado_reconocimientos
    ON modulo6.reconocimientos_iniciales
    IS 'TGR-M06-09e | RF-87, RF-82 | Bloquea INSERT de reconocimientos
iniciales cuando '
        'el período contable está CERRADO o EN_CIERRE.';

-- Aplicar a calculos_valor_razonable
CREATE OR REPLACE TRIGGER tgr_m06_09f_periodo_cerrado_calculos_vr
    BEFORE INSERT OR UPDATE OR DELETE
    ON modulo6.calculos_valor_razonable
    FOR EACH ROW

```

```
EXECUTE FUNCTION modulo6.fn_tgr_m06_09_periodo_cerrado_bloqueo();

COMMENT ON TRIGGER tgr_m06_09f_periodo_cerrado_calculos_vr
ON modulo6.calculos_valor_razonable
IS 'TGR-M06-09f | RF-87, RF-84 Precondición 4 | Bloquea DML sobre
cálculos de valor '
'razonable cuando el período contable está CERRADO o EN_CIERRE.';
```

TGR-M06-10: Inmutabilidad de variaciones de valor razonable (append-only)

Descripción	Impide UPDATE y DELETE directos sobre variaciones_valor_razonable. Las correcciones solo se realizan mediante nuevos registros con id_variacion_corregida. Bloquea cualquier UPDATE o DELETE sobre variaciones_valor_razonable. La única forma de corregir es un nuevo INSERT con id_variacion_corregida referenciando el registro original.
Aplicado a	RF-86
Restricción origen RF	RF-86 — Restricciones: <i>"Los registros de variación son inmutables una vez creados (modelo append-only). Si se detecta un error en un periodo abierto, se crea un registro correctivo con monto opuesto y referencia explícita al registro que corrige."</i> y CA-12: <i>"Los registros de variación son inmutables desde su creación: el sistema bloquea cualquier intento de modificación directa."</i>
Justificación	El modelo append-only de variaciones es central para la auditabilidad NIC 41. La BD debe garantizarlo independientemente de la

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_10_inmutabilidad_variaciones()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M06-10] La variación de valor razonable id=% es
immutable (append-only). '
            'No se permiten modificaciones directas. Para corregir esta
variación, '
            'registre un nuevo asiento correctivo con monto opuesto y '
            'id_variacion_corregida=%. Requiere aprobación del Contador.
',
            'Requerimiento RF-86, Restricciones y CA-12.',
            OLD.id_variacion_valor razonable,
            OLD.id_variacion_valor razonable
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M06-10] La variación de valor razonable id=% no puede
eliminarse. '
            'El modelo append-only garantiza que tanto el registro
original como '
            'el correctivo coexistan en el historial contable. '
            'Requerimiento RF-86, Restricciones y CA-12.',
            OLD.id_variacion_valor razonable
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_10_inmutabilidad_variaciones
BEFORE UPDATE OR DELETE
ON modulo6.variaciones_valor_razonable
FOR EACH ROW
EXECUTE FUNCTION modulo6.fn_tgr_m06_10_inmutabilidad_variaciones();

```



```

COMMENT ON TRIGGER tgr_m06_10_inmutabilidad_variaciones
ON modulo6.variaciones_valor_razonable
IS 'TGR-M06-10 | RF-86 Restricciones (append-only) y CA-12 | Bloquea
UPDATE y DELETE '
'sobre variaciones de valor razonable. Las correcciones se
realizan mediante nuevos '
'registros con id_variacion_corregida referenciando el
original.';

```

TGR-M06-11: Consistencia de monto en variaciones de valor razonable

Descripción	Valida que <code>monto_variacion = variacion_transformacion + variacion_precio_mercado</code> y que <code>monto_variacion != 0</code> (si ambos componentes son válidos). Verifica que la suma de los dos componentes sea igual a <code>monto_variacion</code> con tolerancia de ± 0.01 y que <code>monto_variacion</code> no sea exactamente 0.
Aplicado a	RF-86
Restricción origen RF	RF-86 — Restricciones: <i>"Una variación de valor razonable igual a cero no genera registro contable... si monto_variacion = 0 porque algún valor es nulo o inválido, se trata como fallo."</i> y CA-8: <i>"El sistema registra correctamente los valores variacion_transformacion y variacion_precio_mercado recibidos de RF-83 sin modificarlos."</i> y Proceso Paso 3: validación <code>monto_variacion = variacion_transformacion + variacion_precio_mercado</code>.
Justificación	Es una invariante matemática verificable en la misma fila. La BD puede garantizarla de forma atómica en cada escritura.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_11_consistencia_variacion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_suma_componentes    NUMERIC(18,2);
    v_tolerancia          NUMERIC(18,2) := 0.01;
BEGIN
    -- Validar que monto_variacion no sea cero
    IF NEW.monto_variacion = 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-11] monto_variacion=0 no es válido para un
registro de variación. '
            'Si el valor razonable no cambió (variación real = 0), no
debe generarse '
            'un registro de variación. Requerimiento RF-86,
Restricciones.',
            NEW.monto_variacion
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar que monto = suma de componentes
    v_suma_componentes := NEW.variacion_transformacion +
NEW.variacion_precio_mercado;

    IF ABS(NEW.monto_variacion - v_suma_componentes) > v_tolerancia THEN
        RAISE EXCEPTION
            '[TGR-M06-11] monto_variacion=% no coincide con la suma de
componentes: '
            'variacion_transformacion (%) + variacion_precio_mercado (%)
= %. '
            'Diferencia: %. Tolerancia: %. Requerimiento RF-86, CA-8.',
            NEW.monto_variacion,
            NEW.variacion_transformacion,
            NEW.variacion_precio_mercado,
            v_suma_componentes,
            ABS(NEW.monto_variacion - v_suma_componentes),
            v_tolerancia
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar que valores de referencia sean positivos
    IF NEW.valor_razonable_anterior <= 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-11] valor_razonable_anterior=% debe ser positivo.

```

```

        'Requerimiento RF-86, CA-16.',
        NEW.valor_razonable_anterior
    USING ERRCODE = 'P0001';
END IF;

IF NEW.valor_razonable_nuevo <= 0 THEN
    RAISE EXCEPTION
        '[TGR-M06-11] valor_razonable_nuevo=% debe ser positivo. '
        'Requerimiento RF-86, CA-16.',
        NEW.valor_razonable_nuevo
    USING ERRCODE = 'P0001';
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_11_consistencia_variacion
    BEFORE INSERT
    ON modulo6.variaciones_valor_razonable
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_11_consistencia_variacion();

COMMENT ON TRIGGER tgr_m06_11_consistencia_variacion
    ON modulo6.variaciones_valor_razonable
    IS 'TGR-M06-11 | RF-86 Restricciones, CA-8, CA-14, CA-16 | Valida en
INSERT: '
        '(1) monto_variacion != 0, (2) monto_variacion =
variacion_transformacion + '
        'variacion_precio_mercado con tolerancia ±0.01, (3) valores de
referencia positivos.';

```

TGR-M06-12: Valor contable por costo no negativo en valoraciones_por_costos

Descripción	Garantiza que valor_contable_por_costo nunca sea negativo y que la ecuación
-------------	--

	$\text{valor_contable} = \text{costo_adquisicion} - \text{depreciacion_acumulada} - \text{deterioro_acumulado}$ sea consistente. Antes de INSERT o UPDATE, verifica la ecuación de consistencia y si el $\text{valor_contable_por_costo}$ es negativo lo ajusta automáticamente a 0 (comportamiento definido en el requerimiento).
Aplicado a	RF-88
Restricción origen RF	RF-88 — Restricciones: <i>"El valor contable por costo no puede ser inferior a cero: si la suma de depreciación acumulada y deterioro acumulado supera el costo de adquisición, el valor contable se establece en cero."</i> y CA-3: <i>"El sistema establece el valor contable en cero cuando la suma de depreciación y deterioro supera el costo de adquisición, sin producir valores negativos."</i>
Justificación	Es una restricción matemática sobre datos de la misma fila. La BD puede hacer cumplir que el valor contable nunca sea negativo de forma automática, corrigiéndolo a cero cuando aplique.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_12_valor_contable_no_negativo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_valor_calculado    NUMERIC(18,4);
    v_tolerancia         NUMERIC(18,4) := 0.01;
BEGIN
    -- Validar que costo_adquisicion sea positivo
    IF NEW.costos_adquisicion <= 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-12] costos_adquisicion=% debe ser mayor a cero. '
            'El costo de adquisición es la base del método de valoración
por costo. '
            'Requerimiento RF-88.',

```

```

        NEW.costos_adquisicion
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar que depreciacion y deterioro no sean negativos
    IF NEW.depreciacion_acumulada < 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-12] depreciacion_acumulada=% no puede ser
negativa. '
            'Requerimiento RF-88.',
            NEW.depreciacion_acumulada
        USING ERRCODE = 'P0001';
    END IF;

    IF NEW.deterioro_acumulado < 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-12] deterioro_acumulado=% no puede ser negativo. '
            'Requerimiento RF-88.',
            NEW.deterioro_acumulado
        USING ERRCODE = 'P0001';
    END IF;

    -- Calcular el valor_contable esperado
    v_valor_calculado := NEW.costos_adquisicion
                        - NEW.depreciacion_acumulada
                        - NEW.deterioro_acumulado;

    -- Si es negativo, ajustar a cero (comportamiento NIC 41 Párr. 30)
    IF v_valor_calculado < 0 THEN
        NEW.valor_contable_por_costo := 0;
    ELSE
        -- Validar consistencia con el valor ingresado
        IF ABS(NEW.valor_contable_por_costo - v_valor_calculado) >
v_tolerancia THEN
            RAISE EXCEPTION
                '[TGR-M06-12] valor_contable_por_costo=% no coincide con
',
                '(costos_adquisicion % - depreciacion_acumulada % -
deterioro_acumulado %) = %. '
                'Diferencia: %. Tolerancia: %. Requerimiento RF-88,
CA-2.',
                NEW.valor_contable_por_costo,
                NEW.costos_adquisicion,
                NEW.depreciacion_acumulada,
                NEW.deterioro_acumulado,
                v_valor_calculado,

```

```

        ABS(NEW.valor_contable_por_costo - v_valor_calculado),
        v_tolerancia
    USING ERRCODE = 'P0001';
END IF;
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_12_valor_contable_no_negativo
    BEFORE INSERT OR UPDATE
    ON modulo6.valoraciones_por_costos
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_12_valor_contable_no_negativo();

COMMENT ON TRIGGER tgr_m06_12_valor_contable_no_negativo
    ON modulo6.valoraciones_por_costos
    IS 'TGR-M06-12 | RF-88 Restricciones (valor mínimo cero) y CA-2,
    CA-3 | Valida: '
        '(1) costos_adquisicion > 0, (2) depreciacion y deterioro >= 0, '
        '(3) valor_contable = costo - depreciacion - deterioro; si el
    resultado es negativo '
        'lo ajusta automáticamente a 0 conforme a NIC 41 Párr. 30.';

```

TGR-M06-13: Inmutabilidad de registros de auditoría financiera (append-only)

Descripción	Impide UPDATE y DELETE sobre cualquier registro de auditorias_financieras. El log de auditoría es estrictamente append-only e inmutable. Bloquea absolutamente cualquier UPDATE o DELETE sobre auditorias_financieras. Sin excepciones. Ni el Administrador puede modificar estos registros.
-------------	--

Aplicado a	RF-94
Restricción origen RF	RF-94 — Restricciones, punto 1: <i>"Los registros de RF-94 son append-only e inmutables. No existen operaciones UPDATE ni DELETE sobre ningún registro del repositorio de auditoría, sin excepción. Esto aplica incluso para el Administrador del sistema."</i> y RNF-03: <i>"El repositorio de auditoría debe estar configurado a nivel de motor de base de datos para rechazar operaciones UPDATE y DELETE."</i>
Justificación	El requerimiento lo exige explícitamente a nivel de motor de BD: <i>"La inmutabilidad no puede depender únicamente de la lógica de aplicación."</i> Este es el trigger más crítico del módulo desde una perspectiva de auditoría.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_13_inmutabilidad_auditoria()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M06-13] El registro de auditoría id=% es absolutamente
inmutable. '
            'El repositorio de auditorías financieras es append-only por
mandato del '
            'Estatuto Tributario Colombiano y la NIC 41. Ningún actor
(incluido el Administrador) '
            'puede modificar registros de auditoría. Requerimiento
RF-94, Restricción 1 y RNF-03.',
            OLD.id_auditoria_financiera
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M06-13] El registro de auditoría id=% no puede
eliminarse. '
            'La retención mínima es de 10 años desde el cierre del
período. '
            'El repositorio de auditorías es inmutable por diseño de BD.

```

```

        'Requerimiento RF-94, Restricción 1, RNF-03 y RNF-07.',
        OLD.id_auditoria_financiera
    USING ERRCODE = 'P0001';
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_13_inmutabilidad_auditoria
    BEFORE UPDATE OR DELETE
    ON modulo6.auditorias_financieras
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_13_inmutabilidad_auditoria();

COMMENT ON TRIGGER tgr_m06_13_inmutabilidad_auditoria
    ON modulo6.auditorias_financieras
    IS 'TGR-M06-13 | RF-94 Restricción 1 y RNF-03 | Garantiza
    inmutabilidad absoluta del '
        'log de auditoría. Ningún UPDATE o DELETE está permitido. El
    requerimiento exige '
        'que esta protección sea a nivel de motor de BD, no solo de
    aplicación.';

```

TGR-M06-14: Validación de accounting_account obligatorio en registros_costos y cotizaciones

Descripción	Garantiza que el campo accounting_account no sea un array vacío en registros_costos y cotizaciones. Toda entrada debe tener al menos una cuenta PUC definida. Verifica que accounting_account tenga longitud ≥ 1 y que cada elemento del array no sea nulo ni vacío. Se aplica en INSERT y UPDATE.
Aplicado a	RF-90, RF-COT

Restricción origen RF	RF-90 — Restricción 5.2: "El campo <i>accounting_account</i> es obligatorio para <i>TODOS</i> los registros de RF-90 (<i>MANTENIMIENTO, INVERSIÓN y VENTA</i>). Si el campo está ausente, el registro se rechaza con error <i>CAMPO_CONTABLE_AUSENTE</i>." RF-COT — Restricción 5.2: "El campo es obligatorio. Si el campo está ausente, el registro se rechaza con error <i>CAMPO_CONTABLE_AUSENTE</i> (HTTP 422)."
Justificación	El campo es NOT NULL en el esquema pero puede ser un array vacío {}. La BD puede garantizar que el array tenga al menos un elemento, completando la validación que la restricción NOT NULL no cubre.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_14_accounting_account_obligatorio()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_cuenta TEXT;
BEGIN
    -- Validar que el array no esté vacío
    IF NEW.accounting_account IS NULL OR
array_length(NEW.accounting_account, 1) IS NULL
        OR array_length(NEW.accounting_account, 1) = 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-14] El campo accounting_account es obligatorio y
no puede ser un '
            'array vacío en la tabla %. Debe contener al menos una
cuenta PUC válida. '
            'Requerimiento RF-90 Restricción 5.2 / RF-COT Restricción
5.2.',
            TG_TABLE_NAME
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar que ningún elemento del array sea nulo o cadena vacía
    FOREACH v_cuenta IN ARRAY NEW.accounting_account LOOP
        IF v_cuenta IS NULL OR trim(v_cuenta) = '' THEN
            RAISE EXCEPTION

```

```

        '[TGR-M06-14] El campo accounting_account contiene un
valor nulo o vacío '
        'en la tabla %. Todas las cuentas PUC deben tener un
valor válido. '
        'Requerimiento RF-90 Restricción 5.2.',
        TG_TABLE_NAME
        USING ERRCODE = 'P0001';
    END IF;
END LOOP;

-- Validar que el array no tenga más de 2 elementos (máximo: débito
y crédito)
IF array_length(NEW.accounting_account, 1) > 2 THEN
    RAISE EXCEPTION
        '[TGR-M06-14] El campo accounting_account en la tabla %
contiene % elementos. '
        'El máximo permitido es 2 ([débito, crédito]). '
        'Requerimiento RF-90 Restricción 5.2.',
        TG_TABLE_NAME, array_length(NEW.accounting_account, 1)
        USING ERRCODE = 'P0001';
END IF;

RETURN NEW;
END;
$$;

-- Aplicar a registros_costos
CREATE OR REPLACE TRIGGER tgr_m06_14a_accounting_costos
    BEFORE INSERT OR UPDATE OF accounting_account
    ON modulo6.registros_costos
    FOR EACH ROW
    EXECUTE FUNCTION
modulo6.fn_tgr_m06_14_accounting_account_obligatorio();

COMMENT ON TRIGGER tgr_m06_14a_accounting_costos
    ON modulo6.registros_costos
    IS 'TGR-M06-14a | RF-90 Restricción 5.2 | Valida que
accounting_account tenga '
        '1 o 2 elementos válidos (no vacíos ni nulos).';

-- Aplicar a cotizaciones
CREATE OR REPLACE TRIGGER tgr_m06_14b_accounting_cotizaciones
    BEFORE INSERT OR UPDATE OF accounting_account
    ON modulo6.cotizaciones
    FOR EACH ROW
    EXECUTE FUNCTION

```

```

modulo6.fn_tgr_m06_14_accounting_account_obligatorio();

COMMENT ON TRIGGER tgr_m06_14b_accounting_cotizaciones
ON modulo6.cotizaciones
IS 'TGR-M06-14b | RF-COT Restricción 5.2 | Valida que
accounting_account tenga '
'1 o 2 elementos válidos en cotizaciones.';

```

TGR-M06-15: Validación de monto_costo positivo y estado no vacío en registros_costos

Descripción	Garantiza que <code>monto_costo > 0</code> y que la naturaleza de un costo MANTENIMIENTO nunca pueda reclasificarse como VENTA una vez insertado. En UPDATE, bloquea el cambio de <code>naturaleza_costo</code> de MANTENIMIENTO a cualquier otro valor. En INSERT y UPDATE, verifica que <code>monto_costo > 0</code>.
Aplicado a	RF-90
Restricción origen RF	RF-90 — Restricción 5.1: "Los costos clasificados como MANTENIMIENTO no pueden reclasificarse como VENTA ni capitalizarse en el valor del activo bajo ninguna circunstancia. Esta restricción es absoluta e irrevocable." y descripción: <code>monto_costo > 0</code>.
Justificación	La inmutabilidad de la naturaleza MANTENIMIENTO es una restricción de negocio absoluta que no puede depender del backend. Si la BD no la protege, un UPDATE directo podría reclasificar costos.

```

CREATE OR REPLACE FUNCTION modulo6.fn_tgr_m06_15_costos_restricciones()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Validar monto positivo en INSERT y UPDATE
    IF NEW.monto_costo <= 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-15] monto_costo=% debe ser mayor a cero. '
            'No se admiten costos con valor cero ni negativo.
Requerimiento RF-90.',
            NEW.monto_costo
        USING ERRCODE = 'P0001';
    END IF;

    -- Bloquear reclasificación de MANTENIMIENTO en UPDATE
    IF TG_OP = 'UPDATE' THEN
        IF OLD.naturaleza_costo = 'MANTENIMIENTO'
            AND NEW.naturaleza_costo != 'MANTENIMIENTO' THEN
            RAISE EXCEPTION
                '[TGR-M06-15] El costo id=% tiene naturaleza
MANTENIMIENTO y no puede '
                'reclasificarse como %. Esta restricción es absoluta e
irrevocable. '
                'Requerimiento RF-90, Restricción 5.1.',
                OLD.id_registro_costo, NEW.naturaleza_costo
            USING ERRCODE = 'P0001';
        END IF;

        -- Bloquear reclasificación de VENTA a MANTENIMIENTO también
        IF OLD.naturaleza_costo = 'VENTA'
            AND NEW.naturaleza_costo = 'MANTENIMIENTO' THEN
            RAISE EXCEPTION
                '[TGR-M06-15] El costo id=% tiene naturaleza VENTA y no
puede '
                'reclasificarse como MANTENIMIENTO. Requerimiento RF-90,
Restricción 5.7.',
                OLD.id_registro_costo
            USING ERRCODE = 'P0001';
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

```

```

CREATE OR REPLACE TRIGGER tgr_m06_15_costos_restricciones
  BEFORE INSERT OR UPDATE
  ON modulo6.registros_costos
  FOR EACH ROW
  EXECUTE FUNCTION modulo6.fn_tgr_m06_15_costos_restricciones();

COMMENT ON TRIGGER tgr_m06_15_costos_restricciones
  ON modulo6.registros_costos
  IS 'TGR-M06-15 | RF-90 Restricciones 5.1 y 5.7 | Valida: (1)
monto_costo > 0, '
      '(2) naturaleza MANTENIMIENTO no puede reclasificarse a ningún
otro valor, '
      '(3) naturaleza VENTA no puede reclasificarse a MANTENIMIENTO.';

```

TGR-M06-16: Validación de motivo_revision mínimo en revisiones_reconocimiento

Descripción	Garantiza que motivo_revision tenga al menos 20 caracteres antes de insertar una reversión. Verifica que char_length(trim(motivo_revision)) >= 20 antes de INSERT.
Aplicado a	RF-82
Restricción origen RF	RF-82 — Restricciones, punto 2c: "Motivo obligatorio: el Contador debe ingresar un motivo_reversion (texto libre, mínimo 20 caracteres) que justifique la reversión. Este campo es inmutable una vez confirmada la reversión." y CA-15: "El sistema exige el campo motivo_reversion (mínimo 20 caracteres) antes de confirmar cualquier reversión."
Justificación	Es una validación de longitud de cadena sobre un campo de la misma fila. La BD puede garantizarla con un CHECK o un trigger. El trigger es más informativo que un

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_16_motivo_revision_minimo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.motivo_revision IS NULL OR trim(NEW.motivo_revision) = ''
    THEN
        RAISE EXCEPTION
            '[TGR-M06-16] El campo motivo_revision es obligatorio para
            registrar una reversión. '
            'Debe contener al menos 20 caracteres que justifiquen la
            operación. '
            'Requerimiento RF-82, Restricción 2c y CA-15.'
        USING ERRCODE = 'P0001';
    END IF;

    IF char_length(trim(NEW.motivo_revision)) < 20 THEN
        RAISE EXCEPTION
            '[TGR-M06-16] El motivo_revision tiene % caracteres (mínimo
            requerido: 20). '
            'Ingrese una justificación más detallada de la reversión. '
            'Requerimiento RF-82, Restricción 2c y CA-15.',
            char_length(trim(NEW.motivo_revision))
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_16_motivo_revision_minimo
    BEFORE INSERT
    ON modulo6.revisiones_reconocimiento
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_16_motivo_revision_minimo();

COMMENT ON TRIGGER tgr_m06_16_motivo_revision_minimo
    ON modulo6.revisiones_reconocimiento
    IS 'TGR-M06-16 | RF-82 Restricción 2c y CA-15 | Garantiza que
    motivo_revision '
        'tenga al menos 20 caracteres no vacíos antes de confirmar
    cualquier reversión '

```

```
'de reconocimiento inicial.';
```

TGR-M06-17: Inmutabilidad de cotizaciones en períodos cerrados

Descripción	Bloquea UPDATE y DELETE sobre cotizaciones cuyo período contable está CERRADO. En UPDATE o DELETE, consulta el estado del período de la cotización. Si está CERRADO bloquea la operación.
Aplicado a	RF-COT
Restricción origen RF	RF-COT — Restricción 5.12: <i>"Una vez cerrado el período, la cotización queda inmutable. No se puede modificar ni anular."</i> y FA-10/FA-11: <i>"Rechazo con PERIODO_CONTABLE_CERRADO. HTTP 422. No persiste."</i>
Justificación	Complementa TGR-M06-09 con una verificación específica para cotizaciones. El período puede ya estar cerrado cuando se intente modificar o anular una cotización desde la interfaz.

```
CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_17_cotizacion_periodo_cerrado()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_periodo    modulo6.enum_periodo_contables_estado;
    v_id_periodo        INTEGER;
BEGIN
    IF TG_OP = 'DELETE' THEN
        v_id_periodo := OLD.id_periodo_contable;
    ELSE
        v_id_periodo := OLD.id_periodo_contable; -- El período de origen
```

```

no cambia
    END IF;

    SELECT estado
    INTO v_estado_periodo
    FROM modulo6.periodos_contables
    WHERE id_periodo_contable = v_id_periodo;

    IF v_estado_periodo = 'CERRADO' THEN
        RAISE EXCEPTION
            '[TGR-M06-17] La cotización id=% pertenece al período
contable id=% '
            'que está CERRADO. Las cotizaciones de períodos cerrados son
inmutables: '
            'no pueden modificarse ni anularse. Operación: %. '
            'Requerimiento RF-COT, Restricción 5.12.',
            OLD.id_cotizacion, v_id_periodo, TG_OP
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_17_cotizacion_periodo_cerrado
    BEFORE UPDATE OR DELETE
    ON modulo6.cotizaciones
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_17_cotizacion_periodo_cerrado();

COMMENT ON TRIGGER tgr_m06_17_cotizacion_periodo_cerrado
    ON modulo6.cotizaciones
    IS 'TGR-M06-17 | RF-COT Restricción 5.12, FA-10 y FA-11 | Bloquea
UPDATE y DELETE '
        'sobre cotizaciones cuyo período contable está CERRADO. El
período de emisión '
        'determina la inmutabilidad de la cotización.';

```


TGR-M06-18: Validación de fecha_vencimiento en cotizaciones

Descripción	Garantiza que <code>fecha_vencimiento</code> no sea anterior a <code>fecha_emision</code> cuando se informa. En INSERT y en UPDATE del campo <code>fecha_vencimiento</code>, verifica que si no es nula, sea mayor o igual a <code>fecha_emision</code>.
Aplicado a	RF-COT
Restricción origen RF	RF-COT — Restricción 5.7: <i>"fecha_vencimiento no puede ser anterior a fecha_emision."</i> y FA-5: <i>"Rechazo con FECHA_VENCIMIENTO_INVALIDA (HTTP 422). No persiste."</i> y CA-13.
Justificación	Es una validación de coherencia de fechas dentro de la misma fila, ideal para un trigger o CHECK constraint.

```
CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_18_fecha_vencimiento_cotizacion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha_vencimiento IS NOT NULL
        AND NEW.fecha_vencimiento < NEW.fecha_emision THEN
        RAISE EXCEPTION
            '[TGR-M06-18] fecha_vencimiento=% no puede ser anterior a
fecha_emision=%. '
            'Una cotización no puede vencer antes de ser emitida. '
            'Requerimiento RF-COT, Restricción 5.7 y CA-13.',
            NEW.fecha_vencimiento, NEW.fecha_emision
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar valor_cotizacion_propuesto positivo
    IF NEW."valor_cotizacion_propuesto" <= 0 THEN
        RAISE EXCEPTION
```

```

        '[TGR-M06-18] valor_cotizacion_propuesto=% debe ser mayor a
ceros. '
        'Requerimiento RF-COT, FA-8.',
        NEW."valor_cotizacion_propuesto "
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar valor_razonable_referencia positivo
    IF NEW.valor_razonable_referencia <= 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-18] valor_razonable_referencia=% debe ser mayor a
ceros. '
            'El valor de referencia proviene de RF-86 y debe ser
positivo. '
            'Requerimiento RF-COT, Restricción 5.11.',
            NEW.valor_razonable_referencia
            USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_18_fecha_vencimiento_cotizacion
    BEFORE INSERT OR UPDATE OF fecha_vencimiento,
"valor_cotizacion_propuesto ", valor_razonable_referencia
    ON modulo6.cotizaciones
    FOR EACH ROW
    EXECUTE FUNCTION
modulo6.fn_tgr_m06_18_fecha_vencimiento_cotizacion();

COMMENT ON TRIGGER tgr_m06_18_fecha_vencimiento_cotizacion
    ON modulo6.cotizaciones
    IS 'TGR-M06-18 | RF-COT Restricciones 5.7 y 5.11, FA-5 y FA-8 |
Valida: '
        '(1) fecha_vencimiento >= fecha_emision si se informa, '
        '(2) valor_cotizacion_propuesto > 0, '
        '(3) valor_razonable_referencia > 0.';

```

TGR-M06-19: Bloqueo de campos inmutables en cotizaciones después del registro

Descripción	Impide la modificación de campos que no pueden cambiarse después del registro inicial de una cotización: <code>accounting_account</code>, <code>line_type</code>, <code>activos_cotizados</code>, <code>fecha_emision</code>, <code>type_code</code>, <code>id_periodo_contable</code>. En UPDATE, compara OLD y NEW para los campos protegidos. Si alguno cambió, lanza excepción indicando cuál campo fue el problema.
Aplicado a	RF-COT
Restricción origen RF	RF-COT — Restricción 5.8: <i>"No pueden modificarse: id_cotizacion, fecha_emision, id_periodo_contable, activos_cotizados, accounting_account, LineType, type_code."</i>
Justificación	Son campos de identidad de la cotización. Si el backend no protege estos campos, un UPDATE directo podría alterar la naturaleza del documento AAEF ya generado.

```
CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_19_campos_inmutables_cotizacion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- fecha_emision es inmutable
    IF NEW.fecha_emision != OLD.fecha_emision THEN
        RAISE EXCEPTION
            '[TGR-M06-19] El campo fecha_emision de la cotización id=%
es inmutable '
            'y no puede modificarse. Requerimiento RF-COT, Restricción
5.8.',
            OLD.id_cotizacion
```

```

        USING ERRCODE = 'P0001';
    END IF;

    -- id_perodo_contable es inmutable
    IF NEW.id_perodo_contable != OLD.id_perodo_contable THEN
        RAISE EXCEPTION
            '[TGR-M06-19] El campo id_perodo_contable de la cotización
id=% es inmutable. '
            'Requerimiento RF-COT, Restricción 5.8.',
            OLD.id_cotizacion
        USING ERRCODE = 'P0001';
    END IF;

    -- activos_cotizados es inmutable
    IF NEW.activos_cotizados != OLD.activos_cotizados THEN
        RAISE EXCEPTION
            '[TGR-M06-19] El campo activos_cotizados de la cotización
id=% es inmutable. '
            'Requerimiento RF-COT, Restricción 5.8.',
            OLD.id_cotizacion
        USING ERRCODE = 'P0001';
    END IF;

    -- accounting_account es inmutable
    IF NEW.accounting_account != OLD.accounting_account THEN
        RAISE EXCEPTION
            '[TGR-M06-19] El campo accounting_account de la cotización
id=% es inmutable '
            'después del registro inicial. Requerimiento RF-COT,
Restricción 5.8.',
            OLD.id_cotizacion
        USING ERRCODE = 'P0001';
    END IF;

    -- line_type es inmutable
    IF NEW.line_type != OLD.line_type THEN
        RAISE EXCEPTION
            '[TGR-M06-19] El campo line_type de la cotización id=% es
inmutable. '
            'Requerimiento RF-COT, Restricción 5.8.',
            OLD.id_cotizacion
        USING ERRCODE = 'P0001';
    END IF;

    -- type_code es inmutable
    IF NEW.type_code != OLD.type_code THEN

```

```

        RAISE EXCEPTION
            '[TGR-M06-19] El campo type_code de la cotización id=% es
inmutable '
            'y siempre debe ser "4". Requerimiento RF-COT, Restricción
5.8.',
        OLD.id_cotizacion
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_19_campos_inmutables_cotizacion
    BEFORE UPDATE
    ON modulo6.cotizaciones
    FOR EACH ROW
    EXECUTE FUNCTION
modulo6.fn_tgr_m06_19_campos_inmutables_cotizacion();

COMMENT ON TRIGGER tgr_m06_19_campos_inmutables_cotizacion
    ON modulo6.cotizaciones
    IS 'TGR-M06-19 | RF-COT Restricción 5.8 | Protege los campos de
identidad de la '
        'cotización que no pueden modificarse: fecha_emision,
id_perodo_contable, '
        'activos_cotizados, accounting_account, line_type, type_code.';

```

TGR-M06-20: Impacto económico positivo en deterioros de activos

Descripción	Garantiza que <code>impacto_economico_cop > 0</code> en cada registro de <code>deterorios_activos</code> . En INSERT, verifica que <code>impacto_economico_cop > 0</code> y que la justificación no esté vacía.
Aplicado a	RF-88
Restricción origen RF	RF-88 — Entradas: <i>"impacto_economico_cop: Valor monetario"</i>

	<i>del deterioro asociado al evento sanitario referenciado. Debe ser > 0."</i> y Proceso Paso 5c: <i>"impacto_economico_cop > 0"</i> .
Justificación	Es una validación de valor positivo sobre un campo de la misma fila, apropiada para la BD.

```

CREATE OR REPLACE FUNCTION modulo6.fn_tgr_m06_20_deterioro_positivo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Validar que el impacto económico sea positivo
    IF NEW.impacto_economico_cop <= 0 THEN
        RAISE EXCEPTION
            '[TGR-M06-20] impacto_economico_cop=% debe ser mayor a cero.
        ,
            'El deterioro representa una pérdida de valor documentada;
no puede ser '
            'cero ni negativo. Requerimiento RF-88, Proceso Paso 5c.',
            NEW.impacto_economico_cop
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar que la justificación no esté vacía
    IF NEW.justificacion IS NULL OR trim(NEW.justificacion) = '' THEN
        RAISE EXCEPTION
            '[TGR-M06-20] La justificacion del deterioro es obligatoria.
        ,
            'Debe relacionar el hecho clínico del evento sanitario con
su impacto '
            'monetario sobre el valor del activo. Requerimiento RF-88,
Paso 5a.'
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_20_deterioro_positivo
BEFORE INSERT
ON modulo6.deterorios_activos
FOR EACH ROW
EXECUTE FUNCTION modulo6.fn_tgr_m06_20_deterioro_positivo();

```

```

COMMENT ON TRIGGER tgr_m06_20_deterioro_positivo
ON modulo6.deterioros_activos
IS 'TGR-M06-20 | RF-88 Entradas y Proceso Paso 5a, 5c | Valida que '
    'impacto_economico_cop > 0 y que justificacion no esté vacía al
    registrar '
    'un deterioro de activo biológico.';

```

TGR-M06-21: Inmutabilidad de registros en ejecuciones_cierre_periodo completados o fallidos

Descripción	Impide UPDATE y DELETE sobre ejecuciones de cierre que ya tienen estado COMPLETADO o FALLIDO. Solo los estados EN_PROCESO pueden modificarse (para actualizar el progreso del cierre en curso). En UPDATE, bloquea modificaciones sobre registros en estado COMPLETADO o FALLIDO. En DELETE, bloquea eliminación de cualquier registro independientemente del estado. Permite UPDATE solo sobre estado EN_PROCESO (para actualizar paso_actual, activos_procesados durante la ejecución viva).
Aplicado a	RF-87
Restricción origen RF	RF-87 — Restricciones: <i>"El cierre es una transacción atómica de 9 pasos. Solo existen dos resultados posibles: (a) cierre exitoso... (b) fallo técnico con rollback completo."</i> y <i>"No puede existir más de un proceso de cierre en ejecución simultáneamente."</i> RNF RF-87 Fiabilidad: <i>"La transacción de 9 pasos garantiza que el cierre no deja estados técnicos"</i>

	<i>intermedios."</i>
Justificación	El registro de una ejecución de cierre completada o fallida es evidencia de auditoría. La BD debe garantizar que nadie pueda alterar retroactivamente si un cierre fue exitoso o falló, ni cuántos activos procesó. Esta protección no puede depender del backend.

```

CREATE OR REPLACE FUNCTION modulo6.fn_tgr_m06_21_inmutabilidad_cierre()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M06-21] La ejecución de cierre id=% no puede
eliminar. '
            'Es un registro de auditoría permanente del proceso de
cierre contable. '
            'Estado actual: %. Requerimiento RF-87, RNF Fiabilidad.',
            OLD.id_cierre, OLD.estado
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'UPDATE' AND OLD.estado IN ('COMPLETADO', 'FALLIDO') THEN
        RAISE EXCEPTION
            '[TGR-M06-21] La ejecución de cierre id=% tiene estado % y
es inmutable. '
            'Los cierres completados o fallidos no pueden modificarse
retroactivamente. '
            'Requerimiento RF-87, Restricciones y RNF Fiabilidad.',
            OLD.id_cierre, OLD.estado
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_21_inmutabilidad_cierre
BEFORE UPDATE OR DELETE
ON modulo6.ejecuciones_cierre_periodo
FOR EACH ROW
EXECUTE FUNCTION modulo6.fn_tgr_m06_21_inmutabilidad_cierre();

```



```

COMMENT ON TRIGGER tgr_m06_21_inmutabilidad_cierre
ON modulo6.ejecuciones_cierre_periodo
IS 'TGR-M06-21 | RF-87 Restricciones y RNF Fiabilidad | Garantiza
que '
    'las ejecuciones de cierre en estado COMPLETADO o FALLIDO sean
inmutables. '
    'Bloquea todos los DELETE. Permite UPDATE solo sobre estado
EN_PROCESO.';

```

TGR-M06-22: Unicidad de cierre en ejecución por período contable

Descripción	Impide que existan dos ejecuciones de cierre en estado EN_PROCESO para el mismo período contable simultáneamente. Antes de INSERT o UPDATE que cambie el estado a EN_PROCESO, verifica que no exista otra ejecución en estado EN_PROCESO para el mismo <code>id_periodo_contable</code>. Si existe, lanza excepción.
Aplicado a	RF-87
Restricción origen RF	RF-87 — Restricciones: <i>"No puede existir más de un proceso de cierre en ejecución simultáneamente. El sistema debe implementar bloqueo exclusivo (exclusive lock) sobre el periodo durante la ejecución, para evitar que dos instancias del proceso se ejecuten en paralelo."</i> y FA-5: <i>"HTTP 409: Conflict. Mensaje: 'Conflicto de concurrencia: El periodo está siendo procesado por otro proceso de cierre activo.'"</i>
Justificación	El lock de concurrencia a nivel de aplicación puede fallar en escenarios de alta carga o fallos de red. La BD es el único lugar donde se puede garantizar con certeza absoluta que no haya dos cierres simultáneos para el mismo período.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_22_unicidad_cierre_activo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_cierre_existente INTEGER;
BEGIN
    -- Solo aplica cuando el estado es EN_PROCESO
    IF NEW.estado = 'EN_PROCESO' THEN
        SELECT id_cierre
        INTO v_cierre_existente
        FROM modulo6.ejecuciones_cierre_periodo
        WHERE id_periodo_contable = NEW.id_periodo_contable
            AND estado = 'EN_PROCESO'
            AND id_cierre != COALESCE(NEW.id_cierre, -1)
        LIMIT 1;

        IF FOUND THEN
            RAISE EXCEPTION
                '[TGR-M06-22] El período contable id=% ya tiene una
ejecución de '
                'cierre activa (id_cierre=%). No puede iniciarse otro
proceso de '
                'cierre simultáneo. Requerimiento RF-87, Restricciones
(exclusive lock) y FA-5.',
                NEW.id_periodo_contable, v_cierre_existente
            USING ERRCODE = 'P0001';
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_22_unicidad_cierre_activo
BEFORE INSERT OR UPDATE OF estado
ON modulo6.ejecuciones_cierre_periodo
FOR EACH ROW
EXECUTE FUNCTION modulo6.fn_tgr_m06_22_unicidad_cierre_activo();

COMMENT ON TRIGGER tgr_m06_22_unicidad_cierre_activo
ON modulo6.ejecuciones_cierre_periodo
IS 'TGR-M06-22 | RF-87 Restricciones (exclusive lock) y FA-5 |
Impide que '

```

```
'existan dos ejecuciones de cierre en estado EN_PROCESO para el  
mismo '  
'período contable simultáneamente.';
```

TGR-M06-23: Inmutabilidad de estados de resultados del período

Descripción	Impide UPDATE y DELETE sobre registros de estados_resultados_periodo en estado GENERADO o superior. El estado de resultados de un período es un documento financiero inmutable una vez generado. Bloquea DELETE sobre cualquier estado de resultados. Bloquea UPDATE sobre registros en estado GENERADO o APROBADO. Solo permite UPDATE sobre registros en estados de trabajo internos si los hubiera.
Aplicado a	RF-93
Restricción origen RF	RF-93 — Restricciones, punto 5: <i>"El estado generado es inmutable en su contenido base. Las versiones subsiguientes son regeneraciones independientes del informe... y no modifican el contenido base del período cerrado."</i> y punto 6: <i>"Si alguno de los RF fuente presenta errores críticos... el sistema no debe permitir la generación del estado de resultados hasta que el error sea resuelto."</i>
Justificación	El estado de resultados es el documento financiero central. Una vez que está en estado GENERADO, su contenido base es evidencia contable. La BD debe proteger esta inmutabilidad independientemente de la capa de aplicación.

```
CREATE OR REPLACE FUNCTION  
modulo6.fn_tgr_m06_23_inmutabilidad_estado_resultados()
```

```

RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M06-23] El estado de resultados id=% del período % no
puede '
            'eliminarse. Es un documento financiero permanente. '
            'Requerimiento RF-93, Restricción 5.',
            OLD.id_estado, OLD.id_periodo_contable
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'UPDATE' AND OLD.estado IN ('GENERADO', 'APROBADO') THEN
        RAISE EXCEPTION
            '[TGR-M06-23] El estado de resultados id=% tiene estado % y
es '
            'inmutable en su contenido base. Para versiones
alternativas, '
            'genere un nuevo registro independiente. '
            'Requerimiento RF-93, Restricción 5.',
            OLD.id_estado, OLD.estado
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_23_inmutabilidad_estado_resultados
    BEFORE UPDATE OR DELETE
    ON modulo6.estados_resultados_periodo
    FOR EACH ROW
    EXECUTE FUNCTION
modulo6.fn_tgr_m06_23_inmutabilidad_estado_resultados();

COMMENT ON TRIGGER tgr_m06_23_inmutabilidad_estado_resultados
    ON modulo6.estados_resultados_periodo
    IS 'TGR-M06-23 | RF-93 Restricción 5 | Garantiza inmutabilidad del '
        'estado de resultados en estado GENERADO o APROBADO. Bloquea
todos los DELETE. '
        'Contenido base es permanente; versiones alternativas se crean
como nuevos registros.';

```

TGR-M06-24: Consistencia matemática del resultado neto en estados_resultados_periodo

Descripción	Verifica que $\text{resultado_neto} = \text{total_ingresos_transformacion} + \text{total_ingresos_precio_mercado} + \text{total_ingresos_cosecha} - \text{total_gastos_mantenimiento} - \text{total_bajas_deterioro}$ con tolerancia de ± 0.01 COP. Antes de INSERT o UPDATE, calcula el resultado_neto esperado con los componentes de la fila y verifica que coincida con el valor declarado, con tolerancia de ± 0.01 COP.
Aplicado a	RF-93
Restricción origen RF	RF-93 — Restricciones, punto 2 (REGLA DE CONCILIACIÓN): "El Resultado_Neto calculado debe satisfacer la ecuación: $\text{Resultado_Neto} = \Sigma \text{ganancias RF-86} + \Sigma \text{ingresos RF-85} + \Sigma \text{resultados_positivos RF-45} - \Sigma \text{pérdidas RF-86} - \Sigma \text{costos RF-90} - \Sigma \text{pérdidas_negativas RF-45}$. Cualquier discrepancia bloquea la generación y genera alerta crítica al Contador." y CA-4: "El Resultado_Neto satisface la ecuación de conciliación de la restricción 2 con precisión DECIMAL(18,4)."
Justificación	La ecuación de conciliación es una invariante matemática sobre columnas de la misma fila. La BD puede verificarla atómicamente sin depender del backend, garantizando que ningún estado de resultados con aritmética incorrecta pueda persistirse.

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_24_consistencia_resultado_netto()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_resultado_calculado NUMERIC(18,4);
    v_tolerancia          NUMERIC(18,4) := 0.01;
BEGIN
    -- Calcular resultado neto esperado con los componentes de la fila
    v_resultado_calculado :=
        COALESCE(NEW.total_ingresos_transformacion, 0)
        + COALESCE(NEW.total_ingresos_precio_mercado, 0)
        + COALESCE(NEW.total_ingresos_cosecha, 0)
        - COALESCE(NEW.total_gastos_mantenimiento, 0)
        - COALESCE(NEW.total_bajas_deterioro, 0);

    IF ABS(NEW.resultado_netto - v_resultado_calculado) > v_tolerancia
    THEN
        RAISE EXCEPTION
            '[TGR-M06-24] Inconsistencia en estado de resultados id=%: '
            'resultado_netto=% no coincide con la suma de componentes =
            %. '
            'Ingresos transformacion: %, precio mercado: %, cosecha: %.
            '
            'Gastos mantenimiento: %, bajas deterioro: %. '
            'Diferencia: %. Tolerancia: %. Requerimiento RF-93,
Restricción 2 y CA-4.',
            NEW.id_estado,
            NEW.resultado_netto,
            v_resultado_calculado,
            COALESCE(NEW.total_ingresos_transformacion, 0),
            COALESCE(NEW.total_ingresos_precio_mercado, 0),
            COALESCE(NEW.total_ingresos_cosecha, 0),
            COALESCE(NEW.total_gastos_mantenimiento, 0),
            COALESCE(NEW.total_bajas_deterioro, 0),
            ABS(NEW.resultado_netto - v_resultado_calculado),
            v_tolerancia
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_24_consistencia_resultado_netto

```

```

BEFORE INSERT OR UPDATE
ON modulo6.estados_resultados_periodo
FOR EACH ROW
EXECUTE FUNCTION
modulo6.fn_tgr_m06_24_consistencia_resultado_netto();

COMMENT ON TRIGGER tgr_m06_24_consistencia_resultado_netto
ON modulo6.estados_resultados_periodo
IS 'TGR-M06-24 | RF-93 Restricción 2 (REGLA DE CONCILIACIÓN) y CA-4
| '
    'Valida que resultado_netto = ingresos_transformacion +
ingresos_precio + '
    'ingresos_cosecha - gastos_mantenimiento - bajas_deterioro. '
    'Tolerancia de ±0.01 COP para redondeo.';

```

TGR-M06-25: Inmutabilidad de informes de revelación NIC 41 aprobados

Descripción	Impide UPDATE y DELETE sobre informes de revelación en estado APROBADO o ARCHIVADO. Los informes aprobados son documentos auditables inmutables. Bloquea DELETE sobre cualquier informe. Bloquea UPDATE sobre informes en estado APROBADO o ARCHIVADO. Permite UPDATE solo sobre BORRADOR o RECHAZADO.
Aplicado a	RF-92
Restricción origen RF	RF-92 — Restricciones, punto 5: "Los informes APROBADOS y ARCHIVADOS son inmutables e indisponibles para edición." y punto 4: "Al aprobar una versión, la versión anterior pasa automáticamente a estado ARCHIVADO."
Justificación	Los informes de revelaciones NIC 41 son documentos entregados a auditores,

	entidades crediticias y la UPRA. Una vez aprobados, son evidencia regulatoria. La BD debe proteger su inmutabilidad al mismo nivel que los asientos contables.
--	--

```
CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_25_inmutabilidad_informe_revelacion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M06-25] El informe de revelaciones NIC 41 id=% no
puede eliminarse. '
            'Es un documento regulatorio permanente. Estado actual: %. '
            'Requerimiento RF-92, Restricción 5.',
            OLD.id_informe, OLD.estado
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'UPDATE' AND OLD.estado IN ('APROBADO', 'ARCHIVADO') THEN
        RAISE EXCEPTION
            '[TGR-M06-25] El informe de revelaciones NIC 41 id=% tiene
estado % '
            'y es inmutable. Los informes APROBADOS y ARCHIVADOS no
pueden editarse. '
            'Para una nueva versión, genere un nuevo BORRADOR
independiente. '
            'Requerimiento RF-92, Restricciones 4 y 5.',
            OLD.id_informe, OLD.estado
        USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_25_inmutabilidad_informe_revelacion
BEFORE UPDATE OR DELETE
ON modulo6.informes_revelacion_nic41
FOR EACH ROW
EXECUTE FUNCTION
modulo6.fn_tgr_m06_25_inmutabilidad_informe_revelacion();
```



```

COMMENT ON TRIGGER tgr_m06_25_inmutabilidad_informe_revelacion
ON modulo6.informes_revelacion_nic41
IS 'TGR-M06-25 | RF-92 Restricciones 4 y 5 | Garantiza que los
informes '
'de revelaciones en estado APROBADO o ARCHIVADO sean inmutables.
'
'Bloquea todos los DELETE. Solo BORRADOR y RECHAZADO pueden
modificarse.';

```

TGR-M06-26: Bloqueo de período CERRADO o EN_CIERRE en cadenas_trazabilidad_contable

Descripción	Impide INSERT, UPDATE y DELETE sobre registros de cadenas_trazabilidad_contable vinculados a variaciones de períodos contables CERRADOS. Antes de INSERT/UPDATE/DELETE, navega desde id_variacion_valor_razonable hasta el período contable de esa variación. Si el período está CERRADO, bloquea la operación. Solo aplica cuando existe la FK a una variación.
Aplicado a	RF-91, RF-87
Restricción origen RF	RF-91 — Restricciones, punto 4: <i>"La cadena de trazabilidad de asientos de períodos cerrados es inmutable; no puede ser modificada ni completada retroactivamente."</i> y RF-87 descripción: <i>"Una vez cerrado, ningún registro del periodo puede ser modificado."</i>
Justificación	La cadena de trazabilidad de períodos cerrados es evidencia auditada bajo NIC 41. Si un auditor externo navega la cadena de un período cerrado, debe encontrar exactamente lo que existía al momento del cierre. Esta garantía no puede depender del

```

CREATE OR REPLACE FUNCTION
modulo6.fn_tgr_m06_26_trazabilidad_periodo_cerrado()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_periodo modulo6.enum_periodo_contables_estado;
    v_id_variacion    INTEGER;
BEGIN
    -- Determinar la variación según la operación
    IF TG_OP = 'DELETE' THEN
        v_id_variacion := OLD.id_variacion_valor_razonable;
    ELSE
        v_id_variacion := NEW.id_variacion_valor_razonable;
    END IF;

    -- Solo verificar si existe un vínculo a una variación
    IF v_id_variacion IS NULL THEN
        IF TG_OP = 'DELETE' THEN
            RETURN OLD;
        END IF;
        RETURN NEW;
    END IF;

    -- Obtener el estado del período de la variación vinculada
    SELECT pc.estado
    INTO v_estado_periodo
    FROM modulo6.variaciones_valor_razonable vvr
    JOIN modulo6.periodos_contables pc
    ON pc.id_periodo_contable = vvr.id_periodo_contable
    WHERE vvr.id_variacion_valor_razonable = v_id_variacion;

    IF FOUND AND v_estado_periodo = 'CERRADO' THEN
        RAISE EXCEPTION
            '[TGR-M06-26] La cadena de trazabilidad id=% está vinculada
a la '
            'variación id=% que pertenece a un período contable CERRADO.
',
            'Las cadenas de trazabilidad de períodos cerrados son
inmutables y '
            'no pueden modificarse ni completarse retroactivamente. '
            'Operación: %. Requerimiento RF-91, Restricción 4.',
            COALESCE(NEW.id_asiento, OLD.id_asiento),

```

```

        v_id_variacion, TG_OP
        USING ERRCODE = 'P0001';
    END IF;

    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_26_trazabilidad_periodo_cerrado
    BEFORE INSERT OR UPDATE OR DELETE
    ON modulo6.cadenas_trazabilidad_contable
    FOR EACH ROW
    EXECUTE FUNCTION
modulo6.fn_tgr_m06_26_trazabilidad_periodo_cerrado();

COMMENT ON TRIGGER tgr_m06_26_trazabilidad_periodo_cerrado
    ON modulo6.cadenas_trazabilidad_contable
    IS 'TGR-M06-26 | RF-91 Restricción 4, RF-87 | Garantiza que las
cadenas '
    'de trazabilidad vinculadas a variaciones de períodos CERRADOS
sean inmutables. '
    'No se permite modificar ni completar retroactivamente cadenas de
períodos cerrados.';

```

TGR-M06-27: Validación de tipo_actor en auditorias_financieras

Descripción	Garantiza la exclusividad entre <code>id_usuario</code> e <code>id_sistema</code> según el valor de <code>tipo_actor</code>. Si <code>tipo_actor = USUARIO</code>, <code>id_usuario</code> debe estar presente e <code>id_sistema</code> debe ser nulo (o no existir). La tabla <code>auditorias_financieras</code> ahora tiene <code>tipo_actor</code> como columna explícita. Antes de INSERT, verifica que la
-------------	--

	combinación de tipo_actor con id_usuario sea coherente. Para USUARIO : id_usuario no nulo. Para SISTEMA : id_usuario nulo. Si no se cumple, lanza excepción con detalle del problema.
Aplicado a	RF-94
Restricción origen RF	RF-94 — Restricciones, punto 6: <i>"Todo evento debe tener exactamente una fuente identificada. Las reglas de exclusividad son: tipo_actor = USUARIO → id_usuario obligatorio, id_sistema nulo. tipo_actor = SISTEMA → id_sistema obligatorio, id_usuario nulo. Un evento con ambos campos nulos simultáneamente es inválido."</i> y CA-4: <i>"El sistema rechaza y registra en log técnico cualquier evento con tipo_actor ambiguo."</i>
Justificación	El esquema SQL actualizado ya tiene tipo_actor como columna explícita en auditorias_financieras . La exclusividad usuario/sistema es una invariante de integridad que la BD puede verificar atómicamente. El requerimiento RF-94 lo exige explícitamente a nivel de motor de BD (RNF-03).

```

CREATE OR REPLACE FUNCTION modulo6.fn_tgr_m06_27_tipo_actor_auditoria()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- tipo_actor = USUARIO requiere id_usuario no nulo
    IF NEW.tipo_actor = 'USUARIO' AND NEW.id_usuario IS NULL THEN
        RAISE EXCEPTION
            '[TGR-M06-27] Evento de auditoría inválido:
tipo_actor=USUARIO '
            'requiere id_usuario no nulo. El campo id_usuario no puede
ser '
            'nulo cuando el actor es un usuario humano. '
            'Requerimiento RF-94, Restricción 6 y CA-4.'
        USING ERRCODE = 'P0001';
    END IF;

```

```

-- tipo_actor = SISTEMA requiere id_usuario nulo
IF NEW.tipo_actor = 'SISTEMA' AND NEW.id_usuario IS NOT NULL THEN
    RAISE EXCEPTION
        '[TGR-M06-27] Evento de auditoría inválido:
tipo_actor=SISTEMA '
        'requiere id_usuario nulo. No puede existir un actor humano
'
        'identificado en un evento de proceso automático. '
        'Requerimiento RF-94, Restricción 6 y CA-4.'
    USING ERRCODE = 'P0001';
END IF;

-- El hash_integridad no puede ser nulo ni vacío
IF NEW.hash_integridad IS NULL OR trim(NEW.hash_integridad) = ''
THEN
    RAISE EXCEPTION
        '[TGR-M06-27] El campo hash_integridad es obligatorio en
todo '
        'registro de auditoría financiera. Garantiza la
verificabilidad '
        'del registro ante auditorías externas. '
        'Requerimiento RF-94, Restricción 6 y RNF-04.'
    USING ERRCODE = 'P0001';
END IF;

-- El hash_integridad debe tener exactamente 64 caracteres (SHA-256
hex)
IF char_length(NEW.hash_integridad) != 64 THEN
    RAISE EXCEPTION
        '[TGR-M06-27] El hash_integridad tiene % caracteres; se
esperan '
        'exactamente 64 (SHA-256 hexadecimal). '
        'Requerimiento RF-94, Restricción 4 (hash_integridad).',
        char_length(NEW.hash_integridad)
    USING ERRCODE = 'P0001';
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER tgr_m06_27_tipo_actor_auditoria
    BEFORE INSERT
    ON modulo6.auditorias_financieras
    FOR EACH ROW
    EXECUTE FUNCTION modulo6.fn_tgr_m06_27_tipo_actor_auditoria();

```

```
COMMENT ON TRIGGER tgr_m06_27_tipo_actor_auditoria
ON modulo6.auditorias_financieras
IS 'TGR-M06-27 | RF-94 Restricción 6, CA-4 y RNF-04 | Valida en
INSERT: '
      '(1) tipo_actor=USUARIO requiere id_usuario no nulo, '
      '(2) tipo_actor=SISTEMA requiere id_usuario nulo, '
      '(3) hash_integridad obligatorio y con exactamente 64 caracteres
(SHA-256).';
```